

SafeScape

On-Device Acoustic Distress and Hazard Recognition

A Case Study Comparing Three Deep Learning Architectures

Case Study Report

23CSE473 — Neural Networks and Deep Learning

Submitted by

Amruth Rohan KR (CB.EN.U4ECE23222) — MFCC-CNN

Harish Venkat VS (CB.EN.U4ECE23219) — log-mel CRNN and Distilled Transformer

Under the guidance of Dr. Bagyammal T, Professor, Department of CSE

Amrita School of Computing, Amrita Vishwa Vidyapeetham, Coimbatore

October 2026

Abstract

Personal-safety apps today mostly wait for the user to press a button. In the moments that matter, a person may be unable to do that. SafeScape listens instead: it classifies short, rolling one-second windows of ambient audio into five classes — `distress_call`, `glass_break`, `horn_skid`, `alarm` and `ambience` — and raises an alert when a hazard is heard. The design target is a low-cost phone running fully offline, so model size and CPU latency matter as much as accuracy.

This case study trains and compares three architectures on one dataset of 2,919 public recordings (21,917 windows), split by source file, with test windows whose audio also appears in training removed after a duplicate audit: an MFCC-based CNN, a log-mel CRNN (CNN front-end with a bidirectional GRU), and a tiny Transformer distilled from the CRNN. On the held-out test split the CRNN is the strongest model, with 89.7% accuracy, macro-F1 0.853 and distress-call recall 86.5%; after dynamic int8 quantization it is 516 KB. All three models are served by one FastAPI back-end and compared side by side in a web dashboard, which is also used to test the user interface on held-out clips. Post-hoc, safety-constrained logit calibration and a multi-window inference rule are introduced to keep hazard recall high without retraining.

Keywords: acoustic event detection, CRNN, knowledge distillation, MFCC, log-mel spectrogram, model quantization, public safety, SDG 5, SDG 11

Contents

Abstract	2
1. Introduction	5
1.1 Problem statement	5
1.2 Motivation	5
1.3 Objectives	5
1.4 Contributions	5
2. Literature Survey	5
2.1 MFCC-CNN (surveyed by Amruth Rohan KR)	6
2.2 log-mel CRNN (surveyed by Harish Venkat VS)	6
2.3 Distilled Tiny Transformer (surveyed by Harish Venkat VS)	7
2.4 Standard paper	7
3. Dataset	8
3.1 Pre-processing and split	8
3.2 Duplicate audit	8
4. Methodology	9
4.1 Features	9
4.2 MFCC-CNN (Amruth Rohan KR)	9
4.3 Log-mel CRNN (Harish Venkat VS)	9
4.4 Distilled Transformer (Harish Venkat VS)	9
4.5 Training	10
4.6 Safety-constrained calibration	10
4.7 Compression	10
4.8 Serving rule	11
5. System Design and User Interface	11
5.1 Mobile app	11
5.2 Comparison dashboard	12
6. Results and Comparison	15
6.1 Discussion	16
7. UI Testing with the Deep Learning Models	16
7.1 Functional and edge-case tests	17

8. Limitations and Future Work 18

9. Conclusion..... 18

10. Individual Contributions 18

References 19

1. Introduction

1.1 Problem statement

Harassment, assault and road accidents in public spaces are often preceded or accompanied by characteristic sounds — a scream, breaking glass, a skidding vehicle or a horn held down, an alarm. Existing safety apps rely on a manual SOS trigger, which assumes the user has a free hand, an unlocked phone and the presence of mind to use it. The problem this project addresses is to recognise those sounds automatically, quickly enough to be useful, on hardware an ordinary user already owns, without sending audio to a server.

1.2 Motivation

The project is aligned with UN Sustainable Development Goal 5 (gender equality, through freedom from violence) and Goal 11 (safe and inclusive public spaces). Privacy is a design constraint, not an afterthought: a system that streams microphone audio to the cloud would not be acceptable to most of the people it is meant to protect. That pushes the solution towards small models that run locally, which in turn makes this a natural setting to study the trade-off between accuracy, model size and latency across deep learning architectures.

1.3 Objectives

- Build a five-class acoustic hazard dataset from public sources with a leakage-free split.
- Train one architecture per team member and compare them on identical data and metrics.
- Prioritise hazard recall: a missed scream costs far more than a false alarm.
- Compress the best model for low-cost, on-device inference.
- Integrate every model behind one user interface and test the interface against the models.

1.4 Contributions

- A three-way comparison of MFCC-CNN, log-mel CRNN and a distilled Transformer on the same source-disjoint split, reporting accuracy, macro-F1, hazard recall, size and CPU latency.
- A safety-constrained post-hoc calibration that tunes per-class logit offsets on validation data while forbidding any hazard class from losing recall.
- A multi-window serving rule that lets a hazard in any second of a clip decide the verdict.
- A multi-model inference server and comparison dashboard used for UI testing on held-out clips.
- An audit of the dataset that found and removed byte-identical recordings spanning the split.

2. Literature Survey

Each team member surveyed five recent journal papers (2025–2026) related to the architecture they own. The tables below list each work, what it contributes and its reported result; the paragraph after each table states what SafeScape takes from that line of work.

2.1 MFCC-CNN (surveyed by Amruth Rohan KR)

#	Paper	Venue	Contribution	Reported result
1	Robust Scream Detection and Scream Temporal Interval Prediction Using CNN-Transformer and Windowing CNN — Kim, Jang & Lee	IEEE Access, 13, 71374–71387, 2025	Parallel CNN + Transformer with a regression head predicting exact scream start/end.	~3 pp F-measure gain, ~10× lower EER vs. prior SOTA (relative figures)
2	A lightweight CNN–transformer hybrid architecture with channel attention for real-time hazardous acoustic event detection — Altayeva & Omarov	Frontiers in Artificial Intelligence, 9, art. 1824067, 2026	Lightweight CNN + channel attention for 8-class edge-deployable hazard detection.	93.2% acc · F1 0.86 · 1.35 M params · 0.42 GFLOPs · 14.8 ms/sample
3	Environmental sound classification using two-stream deep neural network with interactive time-frequency attention — Chen & Peng	Applied Acoustics, 238, art. 110794, 2025	Two-stream time/frequency branches with depthwise-separable convolutions.	94.2% on ESC-50 · 95.3% on UrbanSound8K
4	Robust Classification of Urban Sounds in Noisy Environments: SPWVD-MFCC and Dual-Stream Classifier — Peng, Wang & Abdulla	Acoustics Australia, 53(2), 253–268, 2025	Replaces the STFT front-end of MFCC with a Smoothed Pseudo-Wigner–Ville Distribution.	up to 37.2% improvement over STFT-based baselines (relative)
5	Robust Forest Sound Classification Using Pareto-Mordukhovich Optimized MFCC in Environmental Monitoring — Qurthobi, Damaševičius, Barzdaitis & Maskeliūnas	IEEE Access, 13, 20923–20944, 2025	Tunes MFCC extraction parameters rather than fixing them; MobileNet/GoogleNet + BiLSTM.	78.52% average accuracy on FSC22 (27 classes)

MFCCs remain a strong, compact front-end, but the surveyed work shows their weakness under noise and their loss of fine time-frequency structure. SafeScape keeps the MFCC-CNN as the lightest baseline and measures what that compactness costs.

2.2 log-mel CRNN (surveyed by Harish Venkat VS)

#	Paper	Venue	Contribution	Reported result
1	An Ensemble of Convolutional Neural Networks for Sound Event Detection — Mukhamadiyev, Khujayarov, Nabieva & Cho	Mathematics (MDPI), 13(9), art. 1502, 2025	Ensemble CNN + Bi-GRU CRNN for public-safety sound events incl. screams and breaking sounds.	segment-based F1 71.5% · event-based F1 46%
2	Frequency-Adaptive and Multi-Scale CRNN with Hybrid Sequential Modeling for Sound Event Detection — Zhou & Rui	Circuits, Systems, and Signal Processing (Springer), 2026	Frequency-dynamic convolution + sliding multi-scale module + hybrid-minGRU on DESED 2023.	event-based F1 0.636 · PSDS1 0.589 · PSDS2 0.824
3	ASiT-CRNN: sound event detection with fine-tuning of a self-supervised pre-trained ASiT model — Zheng, Zhang, Atito, Yang, Wang & Mei	Digital Signal Processing, 160, art. 105055, 2025	Embeds a self-supervised spectrogram transformer into a CRNN backbone.	PSDS1 0.488 · PSDS2 0.767 vs. CRNN baseline 0.351 / 0.552
4	Dangerous Sound Detection Using Convolutional Feature Extraction and Temporal Modeling with BiLSTM — Omarov & Altayeva	Eng. Technology & Applied Science Research, 15(6), 28850–28855, 2025	CNN + BiLSTM + attention over 8 hazard classes, 1 s window with 0.5 s hop — same framing as SafeScape.	>90% accuracy · 6.8 ms GPU / 23.5 ms CPU per window
5	Environmental sound classification using convolutional recurrent neural network and data	Multimedia Tools and Applications, 84(34), 42827–	CRNN over cepstral features with augmentation, motivated by	99.89% on UrbanSound8K

	augmentation — Bansal & Garg	42849, 2025	security/crime-investigation use.	
--	------------------------------	-------------	-----------------------------------	--

Convolution over log-mel spectrograms followed by a recurrent layer is the recurring design for sound event detection, and the hazard-detection paper by Omarov and Altayeva uses the same 1 s window and 0.5 s hop adopted here. SafeScape's CRNN follows this pattern with a bidirectional GRU.

2.3 Distilled Tiny Transformer (surveyed by Harish Venkat VS)

#	Paper	Venue	Contribution	Reported result
1	CMKD: CNN/Transformer-Based Cross-Model Knowledge Distillation for Audio Classification — Gong, Khurana, Rouditchenko & Glass	IEEE Trans. Pattern Analysis and Machine Intelligence, 48(3), 3571–3585, 2026	CNNs and audio spectrogram transformers are mutually effective teachers; students beat their teachers.	ESC-50 96.8% · student 8 M params vs. AST-Base 88 M (~11× smaller)
2	Parameter-efficient target-specialized audio spectrogram transformers via selective cross-domain knowledge distillation — Liang, Luo & Zhong	Knowledge-Based Systems, 332, art. 114893, 2025/2026	Domain-specific adapters + sparse fusion transfer only useful knowledge into a specialized AST.	matches/outperforms SOTA across six domains, training only a small adapter fraction
3	Acoustic Intelligence with Multi-Stage Model Optimization for Environmental Sound Classification — Sarathchandra, Mallikarachchi, Madushani & Meedeniya	Smart Cities (MDPI), 9(5), art. 86, 2026	Channel pruning → quantization-aware training → distillation, the full compression stack.	511,033 → 50,774 params (~10×) retaining 96.75% accuracy on ESC-10
4	A Sensor-Based TinyML Acoustic Monitoring System for Edge-Side Recognition on Resource-Constrained Microcontrollers — Wang & Yu	Sensors (MDPI), 26(13), art. 3972, 2026	End-to-end MFCC → INT8-quantized NN deployed fully offline on an Arduino-class MCU.	98.28% acc · 97.21% macro-F1 · ~26.9 KB deployed model
5	Low resource multi-head self-attention based transformer for airborne acoustic classification system — Mishra & Ghosh	Journal on Audio, Speech, and Music Processing (Springer), 2026, art. 467	LAiT: inverted residual blocks + compact multi-head self-attention for real-time acoustic classification on low-power devices.	0.23 M params · <1 MB memory · +1.18% over the ULNN lightweight baseline

Knowledge distillation lets a small student approach a larger teacher, and the surveyed TinyML work shows sub-megabyte audio models are practical. SafeScape distils a two-layer Transformer from its own CRNN to test that claim on this task.

2.4 Standard paper

Scream and gunshot detection and localization for audio-surveillance systems — Valenzise, Gerosa, Tagliasacchi, Antonacci & Sarti (Politecnico di Milano), IEEE Conf. on Advanced Video and Signal Based Surveillance (AVSS), pp. 21–26, 2007, DOI 10.1109/AVSS.2007.4425280. It defines the task SafeScape revisits: recognising screams and impulsive hazard sounds in ambient audio for public safety. Where that work used hand-crafted features with Gaussian mixture models on a fixed installation, SafeScape uses learned representations on a personal device.

3. Dataset

No single public dataset covers all five classes, so the dataset is assembled from six sources. Each source category is mapped to one SafeScape class in `scripts/02_build_manifest.py`.

Table 3.1 — Source recordings per class (v2 dataset)

source	distress_call	glass_break	horn_skid	alarm	ambience	total
datasec	0	40	0	0	0	40
esc50	0	40	40	80	966	1126
freesound_cc0	0	33	0	0	0	33
ravdess_supplement	200	0	0	0	0	200
scream_kaggle_1	575	0	0	0	0	575
urbansound8k	0	0	167	435	343	945
total	775	113	207	515	1309	2919

- ESC-50: glass breaking, car horn, siren, clock alarm and a range of ambient categories.
- Kaggle human-screaming detection set: the primary `distress_call` source.
- RAVDESS (Zenodo): angry and fearful speech as a distress-adjacent supplement (200 clips).
- UrbanSound8K: `car_horn` → `horn_skid`, `siren` → `alarm`, and seven everyday urban classes as `ambience` — these are the hard negatives that previously confused the hazard classes.
- DataSEC (Zenodo) and Freesound CC0 previews: additional glass-break recordings.

UrbanSound8K's `gun_shot` class was deliberately excluded. It is a genuine hazard with no class in the five-way taxonomy, and labelling it `ambience` would teach a safety model that gunfire is normal background sound.

3.1 Pre-processing and split

Audio is resampled to 16 kHz mono, denoised with non-stationary spectral gating, and cut into 1.0 s windows with 50 % overlap; windows with RMS energy below $1e-4$ are dropped. The 70/15/15 train/validation/test split is stratified and made at the source-file level, so all windows of a recording stay on one side of the split.

3.2 Duplicate audit

Hashing every source file revealed that the Kaggle scream set ships the same recordings in two folders. Because the split is by file path, 230 test windows — all `distress_call` — were byte-identical to recordings in the training or validation split. Rather than re-split and retrain every model, those windows are flagged (`scripts/18_flag_duplicate_sources.py`) and excluded: every test number in this report is computed on the remaining 2,990 windows, which no model has heard in any form. Validation retains 169 such windows; it is used only for checkpoint selection and calibration, and this is listed as a limitation.

Table 3.2 — Windows per split

class	train	val	test (leak-free)	removed from test
distress_call	3704	782	549	230
glass_break	445	76	76	0
horn_skid	764	137	144	0

alarm	2852	611	613	0
ambience	7741	1585	1608	0

4. Methodology

4.1 Features

Two time-frequency representations are computed with a 512-point FFT and a 10 ms hop (160 samples), giving 101 frames per window. The MFCC-CNN receives 40 MFCCs per frame; the CRNN and the Transformer receive a 64-band log-mel spectrogram. Each feature map is standardised per window (zero mean, unit variance). During training, SpecAugment-style frequency and time masking plus small Gaussian feature noise are applied.

4.2 MFCC-CNN (Amruth Rohan KR)

Four 3×3 convolution blocks (16, 32, 64, 64 channels), each with batch normalisation and ReLU, the first three followed by 2×2 max-pooling, then global average pooling, dropout and a linear classifier. The design treats the 40×101 MFCC matrix as a single-channel image. It has no explicit temporal model; the receptive field of the stacked convolutions is the only way it relates distant frames.

4.3 Log-mel CRNN (Harish Venkat VS)

Two convolution blocks (16 and 32 channels) pool only along frequency (2×1), so the 64 mel bands shrink to 16 while all 101 time steps survive. Each time step's 32×16 feature map is flattened to a 512-dimensional vector and fed to a bidirectional GRU with 128 hidden units per direction. GRU outputs are mean-pooled over time and passed through dropout to a linear layer. Keeping the time axis intact is the point of the design: a glass break lasts a few hundred milliseconds, and the recurrent layer can respond to that burst wherever it falls.

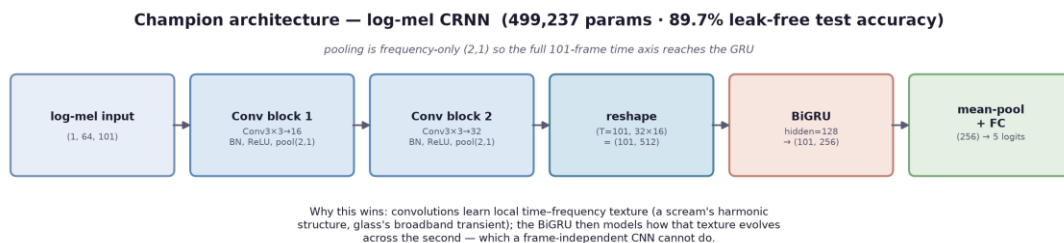


Figure 4.1 — Log-mel CRNN architecture

4.4 Distilled Transformer (Harish Venkat VS)

Each log-mel frame becomes a token: a linear projection maps its 64 bands to a 64-dimensional embedding, a learnable class token is prepended, and learnable positional embeddings are added. Two Transformer encoder layers (4 heads, feed-forward width 128) follow, and the class token's

output feeds a linear classifier. The student is trained against the CRNN teacher with the loss $L = \alpha \cdot \text{CE}(y, s) + (1 - \alpha) \cdot T^2 \cdot \text{KL}(\text{softmax}(t/T) \parallel \text{softmax}(s/T))$, with $\alpha = 0.5$ and temperature $T = 4$, so it learns both the labels and the teacher's relative confidence across classes.

4.5 Training

All models use AdamW with weight decay $1e-4$ and a class-weighted cross-entropy (weights inversely proportional to class frequency), because the hazard classes are much rarer than ambience. The checkpoint kept is the epoch with the best validation macro recall, with early stopping after six epochs without improvement. Hyperparameters for the CNN and CRNN come from an Optuna search over learning rate, batch size and dropout (and GRU width for the CRNN).

Table 4.1 — Hyperparameters

model	learning rate	batch	dropout	other	why
MFCC-CNN	$1.25e-3$	64	0.25	—	Optuna best on validation macro recall; the small model tolerates larger batches
log-mel CRNN	$8.56e-4$	16	0.27	GRU 128 units	Optuna best; small batches and a wider GRU helped the rare classes
Transformer	$1e-3$	32	0.20	$\alpha 0.5, T 4$	standard distillation settings; the teacher already encodes the class balance

4.6 Safety-constrained calibration

Class weighting protects recall but pushes the decision boundary towards the rare classes. Instead of retraining, a per-class bias b is added to the logits before the softmax. b is found by coordinate ascent on the validation split to maximise macro-F1 subject to a floor: no hazard class may lose more than two points of recall relative to the uncalibrated model. An unconstrained search was tried first and rejected — it raised macro-F1 by trading away horn_skid recall, which is the wrong trade for a safety system. Each of the three models is calibrated this way, so they are compared at equivalent operating points.

4.7 Compression

The CRNN is compressed with dynamic post-training quantization to int8, which targets its GRU and linear layers. The MFCC-CNN is convolution-heavy, so dynamic quantization would leave it almost unchanged, and the Transformer is already small; both are served in fp32.

4.8 Serving rule

The app sends two-second clips. The server splits each clip into the same 1 s, 50 %-overlap windows used in training, applies the same energy gate, and classifies every window. If any window's top class is a hazard, the clip takes the most confident hazard window's verdict; otherwise it takes the most confident window. Averaging would let a short hazard be outvoted by the quiet second around it. A clip with no window above the energy floor is reported as ambience, because the model was never trained on silence.

5. System Design and User Interface

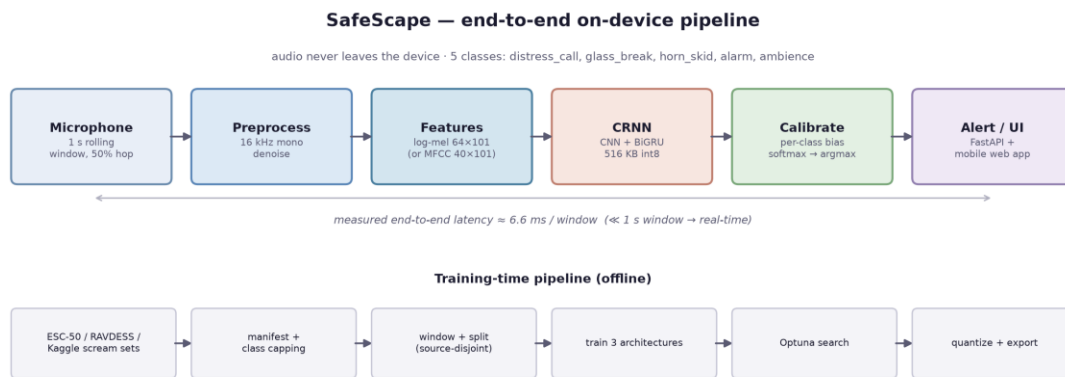


Figure 5.1 — System architecture

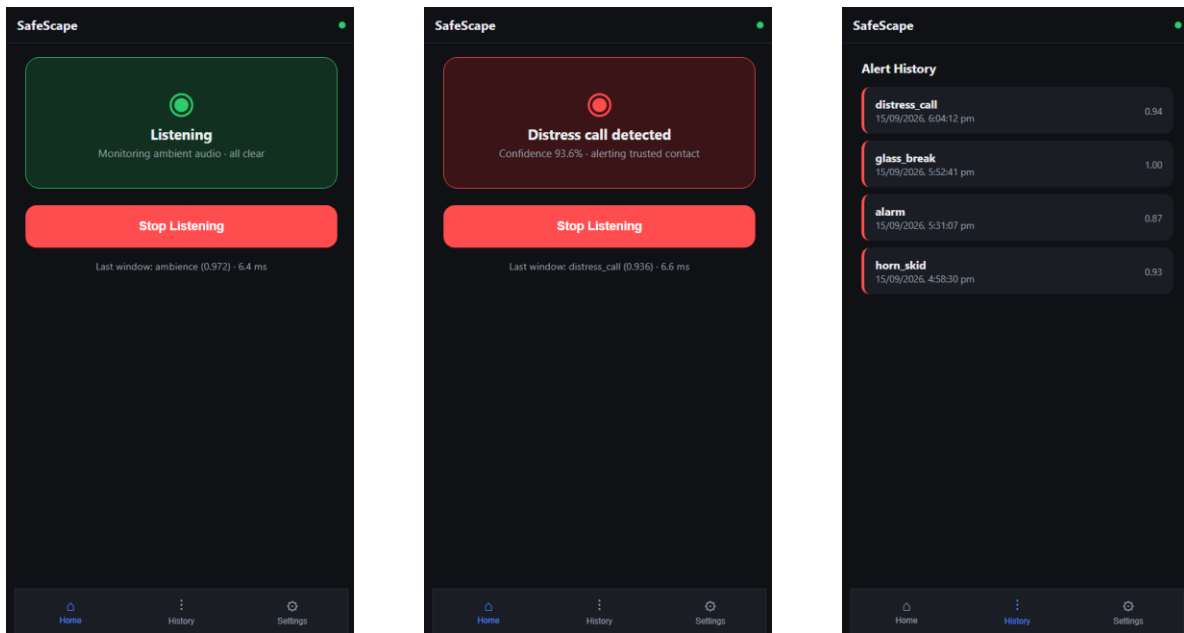
A FastAPI server loads one exported bundle per model (weights, label map, pre-processing configuration and calibration). Browser audio arrives in whatever format the browser records; a bundled static ffmpeg converts it to 16 kHz mono PCM before inference, so the server needs no system-wide audio libraries.

Table 5.1 — Server endpoints

endpoint	purpose
POST /predict?model=<key>	classify a clip with one model (default: CRNN) — used by the mobile app
POST /predict/all	classify one clip with every model and report per-model latency
GET /models	test-set metrics, owner and served size for each model
GET /testclips	the held-out UI test clips and their ground truth
GET /, /compare	mobile app and comparison dashboard

5.1 Mobile app

The mobile web app has three screens. Home shows a large status card that changes colour, icon and text together (so the state never relies on colour alone) between idle, listening and hazard. History lists past alerts with a five-second cool-down so that one event does not flood the list. Settings holds the server address, a connection test, and a link to the comparison dashboard.



listening

hazard detected

alert history

Figure 5.2 — Mobile app screens

5.2 Comparison dashboard

The dashboard is where every model meets the interface. A clip — recorded from the microphone, uploaded, or chosen from the held-out test set — is sent once to `/predict/all`, and three cards show each model's label, confidence, full probability distribution and inference time, with a correct/wrong badge when the ground truth is known. A test-run panel pushes all twenty held-out clips through the same HTTP path the app uses and fills a clip $\times$ model grid with running accuracy. A final panel shows the offline test-set metrics and confusion matrices. The page has no external dependencies, so it works on a laptop with no network.

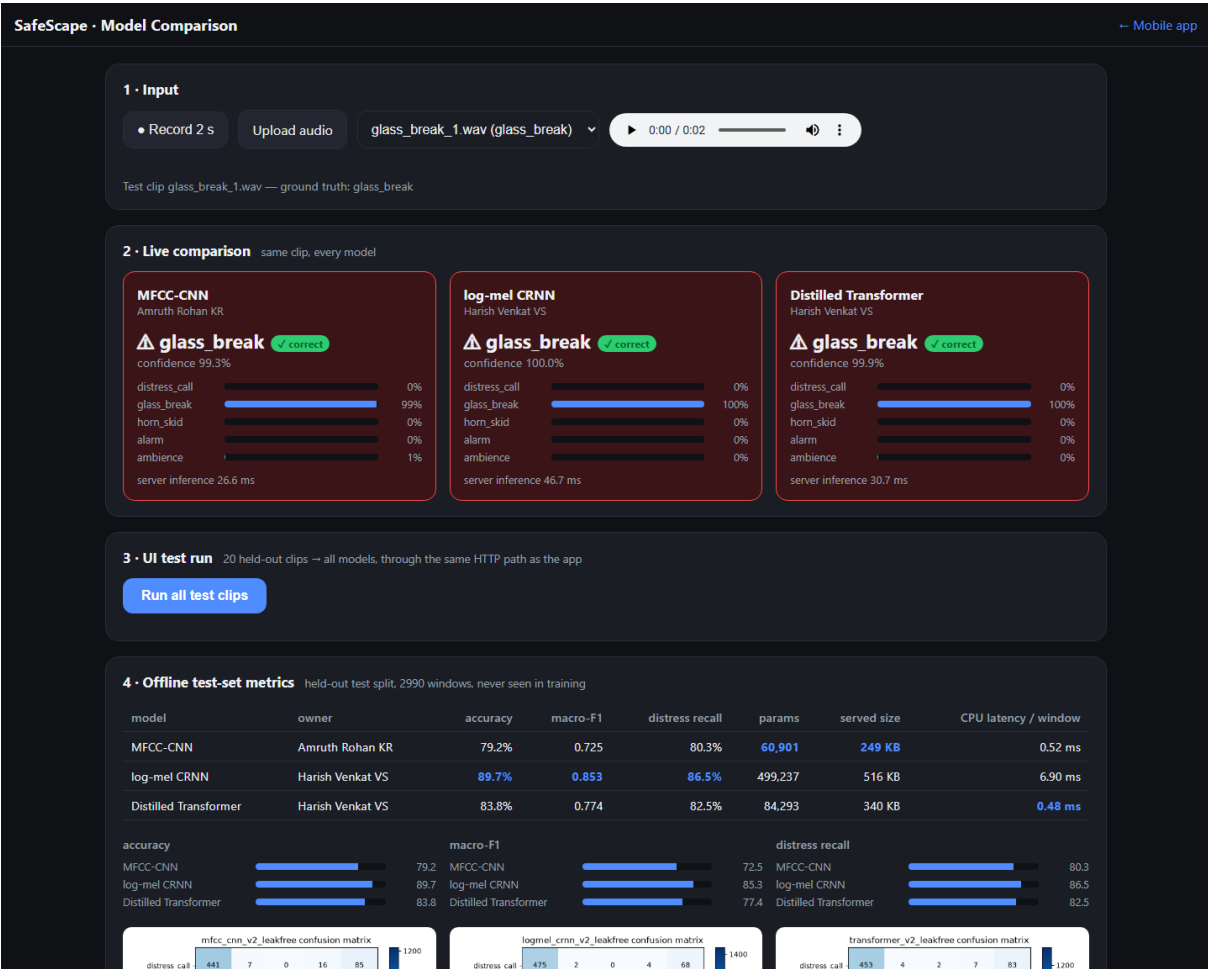


Figure 5.3 — Dashboard: one held-out clip, three models

3 · UI test run 20 held-out clips → all models, through the same HTTP path as the app

Run all test clips

done — 20 clips

clip	truth	MFCC-CNN	log-mel CRNN	Distilled Transformer
distress_call_1.wav	distress_call	✓ distress_call	✓ distress_call	✓ distress_call
distress_call_2.wav	distress_call	✓ distress_call	✓ distress_call	✓ distress_call
distress_call_3.wav	distress_call	✓ distress_call	✓ distress_call	✓ distress_call
distress_call_4.wav	distress_call	✓ distress_call	✓ distress_call	✓ distress_call
glass_break_1.wav	glass_break	✓ glass_break	✓ glass_break	✓ glass_break
glass_break_2.wav	glass_break	✓ glass_break	✓ glass_break	✓ glass_break
glass_break_3.wav	glass_break	✓ glass_break	✓ glass_break	✓ glass_break
glass_break_4.wav	glass_break	✓ glass_break	✓ glass_break	✓ glass_break
horn_skid_1.wav	horn_skid	✓ horn_skid	✓ horn_skid	✓ horn_skid
horn_skid_2.wav	horn_skid	✓ horn_skid	✓ horn_skid	✓ horn_skid
horn_skid_3.wav	horn_skid	✓ horn_skid	✓ horn_skid	✓ horn_skid
horn_skid_4.wav	horn_skid	✗ distress_call	✓ horn_skid	✓ horn_skid
alarm_1.wav	alarm	✓ alarm	✓ alarm	✓ alarm
alarm_2.wav	alarm	✓ alarm	✓ alarm	✓ alarm
alarm_3.wav	alarm	✓ alarm	✓ alarm	✓ alarm
alarm_4.wav	alarm	✓ alarm	✓ alarm	✓ alarm
ambience_1.wav	ambience	✗ alarm	✓ ambience	✓ ambience
ambience_2.wav	ambience	✗ horn_skid	✗ alarm	✗ alarm
ambience_3.wav	ambience	✗ distress_call	✗ alarm	✗ alarm
ambience_4.wav	ambience	✗ distress_call	✗ distress_call	✗ distress_call
accuracy (20 clips)		15/20 · 75.0%	17/20 · 85.0%	17/20 · 85.0%

Figure 5.4 — Dashboard: UI test run over the 20 held-out clips

4 · Offline test-set metrics held-out test split, 2990 windows, never seen in training

model	owner	accuracy	macro-F1	distress recall	params	served size	CPU latency / window
MFCC-CNN	Amruth Rohan KR	79.2%	0.725	80.3%	60,901	249 KB	0.52 ms
log-mel CRNN	Harish Venkat VS	89.7%	0.853	86.5%	499,237	516 KB	6.90 ms
Distilled Transformer	Harish Venkat VS	83.8%	0.774	82.5%	84,293	340 KB	0.48 ms

accuracy			macro-F1				distress recall
MFCC-CNN		79.2	MFCC-CNN		72.5	MFCC-CNN	80.3
log-mel CRNN		89.7	log-mel CRNN		85.3	log-mel CRNN	86.5
Distilled Transformer		83.8	Distilled Transformer		77.4	Distilled Transformer	82.5

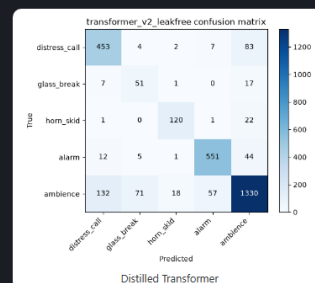
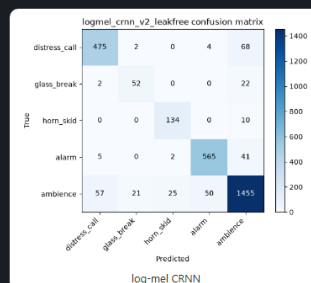
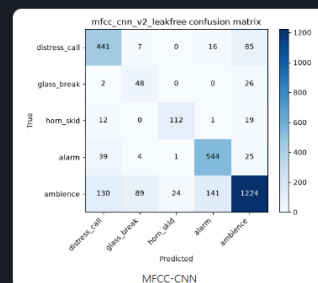


Figure 5.5 — Dashboard: offline test-set metrics

6. Results and Comparison

All figures are on the leak-free held-out test split (2,990 windows), with each model's safety-constrained calibration applied. Latency is the mean time for one 1 s window on the CPU, batch size 1.

Table 6.1 — Three-model comparison

model	owner	accuracy	macro-F1	macro recall	distress recall	params	served size	CPU latency
MFCC-CNN	Amruth	79.2%	0.725	77.2%	80.3%	60,901	249 KB	0.52 ms
log-mel CRNN	Harish	89.7%	0.853	86.1%	86.5%	499,237	516 KB	6.90 ms
Distilled Transformer	Harish	83.8%	0.774	81.1%	82.5%	84,293	340 KB	0.48 ms

Table 6.2 — Per-class precision (P) and recall (R)

class	MFCC-CNN P	MFCC-CNN R	CRNN P	CRNN R	Transformer P	Transformer R
distress call	0.71	0.80	0.88	0.87	0.75	0.83
glass break	0.32	0.63	0.69	0.68	0.39	0.67
horn skid	0.82	0.78	0.83	0.93	0.85	0.83
alarm	0.77	0.89	0.91	0.92	0.89	0.90
ambience	0.89	0.76	0.91	0.90	0.89	0.83

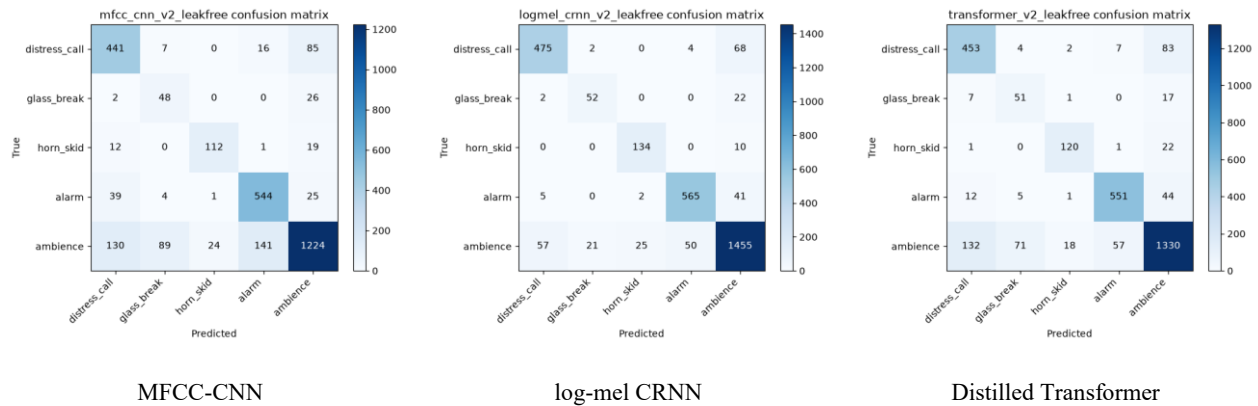


Figure 6.1 — Confusion matrices on the leak-free test split (rows: true class)

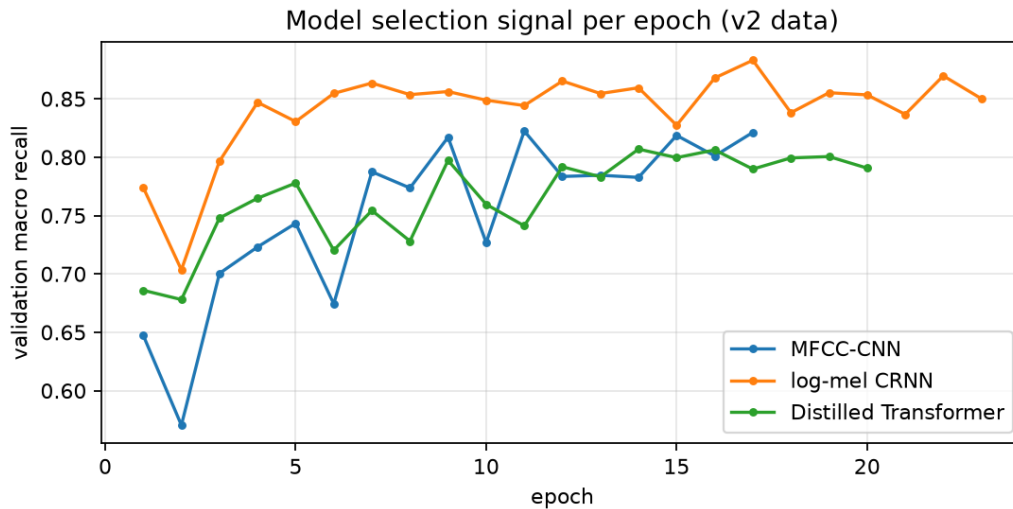


Figure 6.2 — Validation macro recall per epoch

6.1 Discussion

The CRNN leads on every quality metric (macro-F1 0.853 against 0.774 for the Transformer and 0.725 for the MFCC-CNN). Two design choices account for most of the gap. Log-mel features keep the spectral detail that the MFCC's cosine transform compresses away, and the CRNN's recurrent layer models how a sound evolves over the window, where the CNN can only pool over it.

The distilled Transformer has 84,293 parameters against the CRNN's 499,237 and runs in 0.48 ms per window against 6.90 ms. Distillation transfers much, but not all, of the teacher's behaviour; on a dataset of this size the teacher's convolutional inductive bias still wins.

The MFCC-CNN is the smallest and fastest model (60,901 parameters, 0.52 ms). It also gained the most from the expanded dataset: its accuracy was 66.6% on the first dataset version and is now 79.2%. The two test splits differ, so the comparison is indicative, but it suggests the earlier results were limited by data more than by the architecture.

The deployment choice follows from the safety requirement. All three models are fast enough for real time — each is far below the one-second window length — so the deciding factor is hazard recall. The quantized CRNN, at 516 KB, is small enough for any phone and is the default model in the app.

7. UI Testing with the Deep Learning Models

The user interface was tested against all three models with twenty clips — four per class — cut from test-split recordings that the duplicate audit confirmed no model has heard. Each clip is the two seconds around the loudest second of its source, the same length the app records. The table below is produced by running the served bundles on those clips; the dashboard's test run produces the same grid through HTTP.

Table 7.1 — UI test results (held-out clips)

clip	truth	MFCC-CNN	CRNN	Transformer
distress_call_1.wav	distress_call	✓ distress_call	✓ distress_call	✓ distress_call
distress_call_2.wav	distress_call	✓ distress_call	✓ distress_call	✓ distress_call
distress_call_3.wav	distress_call	✓ distress_call	✓ distress_call	✓ distress_call
distress_call_4.wav	distress_call	✓ distress_call	✓ distress_call	✓ distress_call
glass_break_1.wav	glass_break	✓ glass_break	✓ glass_break	✓ glass_break
glass_break_2.wav	glass_break	✓ glass_break	✓ glass_break	✓ glass_break
glass_break_3.wav	glass_break	✓ glass_break	✓ glass_break	✓ glass_break
glass_break_4.wav	glass_break	✓ glass_break	✓ glass_break	✓ glass_break
horn_skid_1.wav	horn_skid	✓ horn_skid	✓ horn_skid	✓ horn_skid
horn_skid_2.wav	horn_skid	✓ horn_skid	✓ horn_skid	✓ horn_skid
horn_skid_3.wav	horn_skid	✓ horn_skid	✓ horn_skid	✓ horn_skid
horn_skid_4.wav	horn_skid	X distress_call	✓ horn_skid	✓ horn_skid
alarm_1.wav	alarm	✓ alarm	✓ alarm	✓ alarm
alarm_2.wav	alarm	✓ alarm	✓ alarm	✓ alarm
alarm_3.wav	alarm	✓ alarm	✓ alarm	✓ alarm
alarm_4.wav	alarm	✓ alarm	✓ alarm	✓ alarm
ambience_1.wav	ambience	X alarm	✓ ambience	✓ ambience
ambience_2.wav	ambience	X horn_skid	X alarm	X alarm
ambience_3.wav	ambience	X distress_call	X alarm	X alarm
ambience_4.wav	ambience	X distress_call	X distress_call	X distress_call
accuracy		15/20	17/20	17/20

7.1 Functional and edge-case tests

Table 7.2 — Functional tests

test	expected	result
Hazard in the first or second half of a clip, silence in the other	hazard detected	pass for all four hazard classes (server/test_clip_position.py)
Ambience followed by one second of silence	ambience	fail — distress_call (0.77); identical with the earlier model, see Section 8
Two seconds of silence	ambience from every model	pass (server/test_multi_model.py)
0.33 s clip	every model answers	pass (server/test_multi_model.py)
Unknown model key	HTTP 404, server keeps running	pass
Dashboard opened without microphone access	record disabled with reason; upload and test clips still work	pass
Existing mobile app against the new server	unchanged behaviour on /predict	pass (server/test_client.py)

Every model recognised all sixteen hazard clips except one horn clip that the MFCC-CNN called a distress call. The failures are concentrated in ambience. This is the cost of the serving rule in Section 4.8, which lets any single window classified as a hazard decide the whole clip: a deliberate bias towards false alarms over missed events. The test clips also make it harder than everyday use, because each one is cut around the loudest second of its recording, so a background recording is represented by its most event-like moment. Window-level ambience recall on the offline test split is much higher (Table 6.2), and a practical refinement is to require a hazard in two consecutive windows before alerting.

The clip-position test exposed a real defect during development: the server originally kept only the last second of each two-second clip, so a glass break in the first half was misclassified. The multi-window rule in Section 4.8 was the fix, and the test now guards it.

8. Limitations and Future Work

- Distress audio comes from acted or emotional-speech recordings, not real incidents or India-specific environments; a consented, self-recorded set is the most valuable next step.
- 169 validation windows still duplicate training audio; re-splitting on content hashes and retraining would remove the last of the leak.
- `glass_break` remains the thinnest class (597 windows) and the hardest one to get right.
- Ambience followed by silence is misread as `distress_call`. The earlier model behaves the same, which points at pre-processing rather than the retrained weights; a likely cause, not yet confirmed, is per-window standardisation stretching the quiet tail to unit variance.
- The any-hazard-window rule favours recall over precision; requiring agreement between consecutive windows would cut false alarms on busy background sound.
- Inference runs on a laptop server reachable over the LAN, not on the phone itself; exporting to TorchScript or ONNX for mobile runtimes is the next engineering step.
- Browsers allow microphone access only on HTTPS or localhost, so live phone capture needs a certificate or a native wrapper.
- Static quantization or quantization-aware training would let the convolutional layers be compressed as well.

9. Conclusion

SafeScape shows that a small, fully offline model can recognise acoustic hazards with useful accuracy. The log-mel CRNN reached 89.7% accuracy and 86.5% distress-call recall on audio no model had heard, in 516 KB after quantization. The comparison suggests that, for short acoustic events, keeping the time axis and modelling it explicitly matters more than raw model size, and that data coverage of the rare classes mattered more than any architectural change. Integrating all three models behind one interface made these differences visible clip by clip, and testing that interface surfaced a serving defect that offline metrics alone would have missed.

10. Individual Contributions

member	contributions
Amruth Rohan KR (CB.EN.U4ECE23222)	MFCC feature pipeline; MFCC-CNN design, hyperparameter search and training on v1 and v2 data; MFCC-CNN evaluation and calibration; literature survey for the CNN track.
Harish Venkat VS (CB.EN.U4ECE23219)	Dataset assembly and audit; log-mel CRNN design, search, training, calibration and quantization; Transformer distillation; FastAPI server, mobile app and comparison dashboard; literature survey for the CRNN and Transformer tracks.

References

- [1] Valenzise, Gerosa, Tagliasacchi, Antonacci & Sarti (Politecnico di Milano), "Scream and gunshot detection and localization for audio-surveillance systems," IEEE Conf. on Advanced Video and Signal Based Surveillance (AVSS), pp. 21–26, 2007, doi: 10.1109/AVSS.2007.4425280.
- [2] Kim, Jang & Lee, "Robust Scream Detection and Scream Temporal Interval Prediction Using CNN-Transformer and Windowing CNN," IEEE Access, 13, 71374–71387, 2025, doi: 10.1109/ACCESS.2025.3556729.
- [3] Altayeva & Omarov, "A lightweight CNN–transformer hybrid architecture with channel attention for real-time hazardous acoustic event detection," Frontiers in Artificial Intelligence, 9, art. 1824067, 2026, doi: 10.3389/frai.2026.1824067.
- [4] Chen & Peng, "Environmental sound classification using two-stream deep neural network with interactive time-frequency attention," Applied Acoustics, 238, art. 110794, 2025, doi: 10.1016/j.apacoust.2025.110794.
- [5] Peng, Wang & Abdulla, "Robust Classification of Urban Sounds in Noisy Environments: SPWVD-MFCC and Dual-Stream Classifier," Acoustics Australia, 53(2), 253–268, 2025, doi: 10.1007/s40857-025-00350-6.
- [6] Qurthobi, Damaševičius, Barzdaitis & Maskeliūnas, "Robust Forest Sound Classification Using Pareto-Mordukhovich Optimized MFCC in Environmental Monitoring," IEEE Access, 13, 20923–20944, 2025, doi: 10.1109/ACCESS.2025.3535796.
- [7] Mukhamadiyev, Khujayarov, Nabieva & Cho, "An Ensemble of Convolutional Neural Networks for Sound Event Detection," Mathematics (MDPI), 13(9), art. 1502, 2025, doi: 10.3390/math13091502.
- [8] Zhou & Rui, "Frequency-Adaptive and Multi-Scale CRNN with Hybrid Sequential Modeling for Sound Event Detection," Circuits, Systems, and Signal Processing (Springer), 2026, doi: 10.1007/s00034-026-03754-5.
- [9] Zheng, Zhang, Atito, Yang, Wang & Mei, "ASiT-CRNN: sound event detection with fine-tuning of a self-supervised pre-trained ASiT model," Digital Signal Processing, 160, art. 105055, 2025, doi: 10.1016/j.dsp.2025.105055.
- [10] Omarov & Altayeva, "Dangerous Sound Detection Using Convolutional Feature Extraction and Temporal Modeling with BiLSTM," Eng. Technology & Applied Science Research, 15(6), 28850–28855, 2025, doi: 10.48084/etasr.13068.
- [11] Bansal & Garg, "Environmental sound classification using convolutional recurrent neural network and data augmentation," Multimedia Tools and Applications, 84(34), 42827–42849, 2025, doi: 10.1007/s11042-025-20820-3.
- [12] Gong, Khurana, Rouditchenko & Glass, "CMKD: CNN/Transformer-Based Cross-Model Knowledge Distillation for Audio Classification," IEEE Trans. Pattern Analysis and Machine Intelligence, 48(3), 3571–3585, 2026, doi: 10.1109/TPAMI.2025.3644853.
- [13] Liang, Luo & Zhong, "Parameter-efficient target-specialized audio spectrogram transformers via selective cross-domain knowledge distillation," Knowledge-Based Systems, 332, art. 114893, 2025/2026, doi: 10.1016/j.knosys.2025.114893.
- [14] Sarathchandra, Mallikarachchi, Madushani & Meedeniya, "Acoustic Intelligence with Multi-Stage Model Optimization for Environmental Sound Classification," Smart Cities (MDPI), 9(5), art. 86, 2026, doi: 10.3390/smartcities9050086.
- [15] Wang & Yu, "A Sensor-Based TinyML Acoustic Monitoring System for Edge-Side Recognition on Resource-Constrained Microcontrollers," Sensors (MDPI), 26(13), art. 3972, 2026, doi: 10.3390/s26133972.

- [16] Mishra & Ghosh, "Low resource multi-head self-attention based transformer for airborne acoustic classification system," *Journal on Audio, Speech, and Music Processing (Springer)*, 2026, art. 467, doi: 10.1186/s13636-026-00467-0.
- [17] K. J. Piczak, "ESC: Dataset for Environmental Sound Classification," *Proc. 23rd ACM Int. Conf. on Multimedia*, pp. 1015–1018, 2015.
- [18] J. Salamon, C. Jacoby and J. P. Bello, "A Dataset and Taxonomy for Urban Sound Research," *Proc. 22nd ACM Int. Conf. on Multimedia*, pp. 1041–1044, 2014.
- [19] S. R. Livingstone and F. A. Russo, "The Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS)," *PLoS ONE*, 13(5), e0196391, 2018.
- [20] D. S. Park et al., "SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition," *Proc. Interspeech*, pp. 2613–2617, 2019.
- [21] G. Hinton, O. Vinyals and J. Dean, "Distilling the Knowledge in a Neural Network," *NIPS Deep Learning Workshop*, 2015.
- [22] T. Akiba et al., "Optuna: A Next-generation Hyperparameter Optimization Framework," *Proc. 25th ACM SIGKDD*, pp. 2623–2631, 2019.